
R 语言自学笔记

R



魏舟

2026 年 2 月 12 日

自己捋饬 R 语言的笔记

Contents

1	R 语言简介	5
1.1	安装网址	5
1.2	界面操作示例	6
1.3	一个简单的示例 (不是 Hello World)	6
2	R 语言预备知识	7
2.1	基本命令	7
2.1.1	获取工作目录	7
2.1.2	设置工作目录	7
2.1.3	注释	7
2.1.4	变量名的命名	8
2.1.5	赋值	8
2.1.6	变量的显示	8
2.1.7	查看与清除变量	8
2.1.8	函数帮助文档查询	9
2.1.9	软件的退出与保存	9
2.1.10	保存数据	9
2.2	R 语言语法	9
2.2.1	函数	9
2.2.2	向量 (Vector)	13
2.2.3	NA 与 NULL 值	15
2.2.4	矩阵 (Matrix)	17
2.2.5	列表 (List)	18
2.2.6	数据框 (Data Frame)	19
2.2.7	因子 (Factor)	21
2.2.8	常用数学与统计函数	22
2.3	控制流与自定义函数	23
3	数据分析	25
3.1	数据清洗与转换 (dplyr & tidyr)	25
3.1.1	dplyr 的核心动作 (Verbs)	25

3.1.2	tidyr 的数据重塑	25
3.1.3	常用清洗函数对比总结	25
3.2	数据可视化 (ggplot2)	25
3.2.1	ggplot2 的基础语法结构	26
3.2.2	常见图表类型 (Geoms)	26
3.2.3	图形美化: 颜色、标签与主题	26
3.2.4	分面 (Faceting)	27
3.2.5	常用外观调整函数对照表	27
3.3	统计检验基础	28
3.3.1	均值比较 (T-Test & ANOVA)	28
3.3.2	相关性与回归分析	28
3.3.3	常用统计函数速查表	29

Chapter 1

R 语言简介

R 语言不仅是一门编程语言，更是数据科学领域处理复杂统计逻辑的专业环境。其核心竞争力在于将严谨的数学模型与出版级的数据可视化（Data Visualization）完美融合。不同于通用编程语言，R 的设计哲学高度契合数学直觉，例如在处理标准正态分布 $N(0, 1)$ 的概率密度计算时：

$$f(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}} \quad (1.1)$$

开发者仅需调用 `dnorm(x)` 即可实现。

在图形生成方面，R 的 `ggplot2` 扩展包基于“图形语法”理论，允许用户通过图层叠加的方式构建极其复杂的视觉表达。如下代码展示了如何快速生成带有线性拟合带的专业散点图：

```
1 library(ggplot2)
2 # 使用内置数据集绘制：散点 + 线性回归趋势线
3 ggplot(mtcars, aes(x=wt, y=mpg)) +
4   geom_point(color="steelblue", size=2) +
5   geom_smooth(method="lm", se=TRUE, col="indianred") +
6   theme_minimal() +
7   labs(title="Weight vs. MPG Regression", x="Weight", y="Miles/Gallon")
```

通过这种“声明式”的编程方式，R 极大地缩短了从原始数据到学术洞察的路径。无论是进行大规模基因序列分析，还是金融市场的波动率建模，R 凭借 CRAN 镜像站中超过两万个专业扩展包的支持，始终占据着统计科研领域的统治地位。

1.1 安装网址

网址如下，

R 语言安装网址

<https://cran.r-project.org/>

IDE 安装网址

<https://posit.co/download/rstudio-desktop/>

注意安装路径选择全英文

1.2 界面操作示例

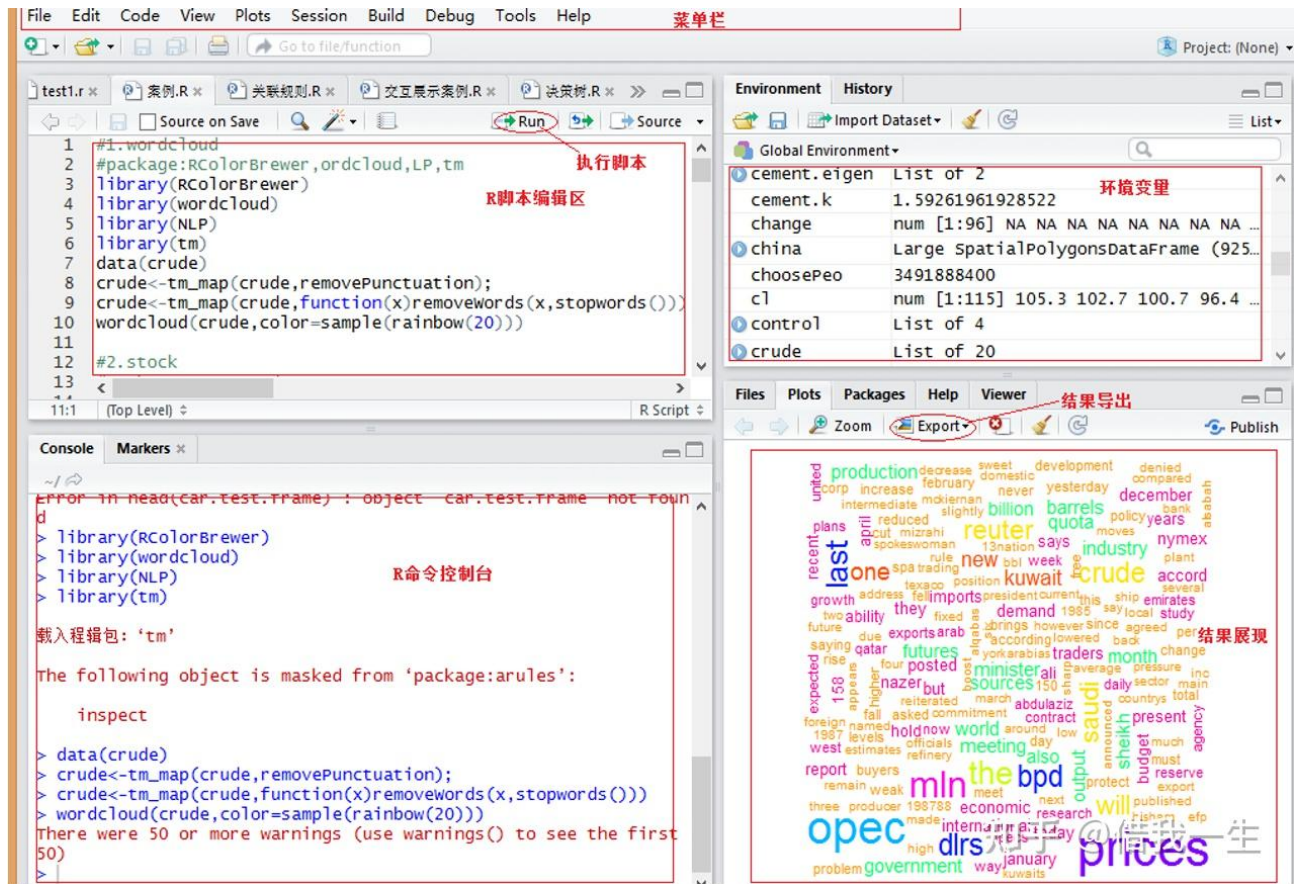


Figure 1.1: R studio 操作界面

1.3 一个简单的示例 (不是 Hello World)

```

1 # Listing 1.1 - A Sample R session
2 age <- c(1,3,5,2,11,9,3,9,12,3)
3 weight <- c(4.4,5.3,7.2,5.2,8.5,7.3,6.0,10.4,10.2,6.1)
4 mean(weight)
5 sd(weight)
6 cor(age,weight)
7 plot(age,weight)

```

Chapter 2

R 语言预备知识

2.1 基本命令

2.1.1 获取工作目录

使用 `getwd()` 函数获取当前工作目录，作用是查看当前工作目录在哪一个具体位置，C 盘或者是其他，第一次使用可能会出现警告，但是也可以看到工作目录，你可以在运行一次代码，如下：

```
1 > getwd()
2 [1] "C:/Users/橙/Documents"
3 Warning message:
4 In normalizePath(path.expand(path), winslash, mustWork) :
5   path[1]="C:/Users/?/Documents": 文件名、目录名或卷标语法不正确。
6 > getwd()
7 [1] "C:/Users/橙/Documents"
```

2.1.2 设置工作目录

使用 `setwd()` 函数更改当前目录，把工作目录更改到你存放你需要使用的数据包的那个位置，更便于直接重工作目录中读取，否则需要使用全路径。在使用函数是需要几点注意，请看以下代码：

```
1 > setwd("F:\\R.cx")#工作目录的地址需要用双引号括起来，注意斜杠，这种单斜杠会报错
2 Error: '\\R' is an unrecognized escape in character string starting '"F:\\R"'
3 > setwd("F:/R.cx")这种单斜杠可以，能运行#
4 > setwd("F://R.cx")这行与下一行的双斜杠都行#
5 > setwd("F:\\R.cx")
6 >
```

2.1.3 注释

R 语言的注释使用 `#` 号，写代码一定要常写注释，不然时间久了会忘记某代码的作用，便于后续操作。在 Rstudio 中可以使用 `Ctrl+shift+C` 进行多行注释。

2.1.4 变量名的命名

与其他语言的变量名命名差不多，主要要注意：数字不能开头；% 号是非法字符，不可以用来命名；. 号后面不可以跟数字；不可以下划线开头。变量名可以为与你操作相关的名字命名，如我要对一行身高的数据求均值：

```
1 high<-c(175,169,179,175,180,183)
2 high_mean<-mean(high)#对身高求均值
```

2.1.5 赋值

R 语言的赋值方式与其他语言有点区别，有三种，分别是左箭头 <- (< 键 +-键 (等号左边的那个，不要按 Shift))，等号 =，右箭头 ->，如：

```
1 > a<-2
2 > b=3
3 > c->4 #注意右箭头使用方法，被赋值的应该在右边，简单点说，就是某某赋值给谁，箭头就指向谁
4 Error in 4 <- c : invalid (do_set) left-hand side to assignment
5 > 4->c
6 > a #键盘敲出变量名，回车后就会显示结果
7 [1] 2
8 > b
9 [1] 3
10 > c
11 [1] 4
12 >
```

2.1.6 变量的显示

1. 命令行直接输入变量名

2. 利用 print() 函数，如下

```
1 > a<-2
2 > a #命令行直接输入变量名
3 [1] 2
4 > print(a) #利用print()函数
5 [1] 2
```

2.1.7 查看与清除变量

查看变量：ls()

清除指定变量：rm()

清除所有变量：`rm(list = ls())` 如下：

```
1 > ls()      #查看变量，这个函数是查看所有已定义的变量，输出所有变量名
2 [1] "a" "b" "c"
3 > rm(a)     #清除变量a
4 > rm(list=ls()) #清除所有变量
```

2.1.8 函数帮助文档查询

当遇到一个函数不会用时，可以使用帮助文档查询，结果会在 Rstudio 的右下角框里显示，方法有三种：

1. ?+ 要查询的函数
2. 使用 `help()` 函数查询
3. 使用 `example()` 函数查询

我以求和函数 `sum()` 为例：

```
1 > ?sum
2 > help(sum)
3 > example(sum)
```

2.1.9 软件的退出与保存

1. 可以直接输入 `q()` 函数；也可以右上角直接叉掉，后根据提示操作
2. 当写完一个文件时，可以点击保存按钮，根据提示选择保存位置，文件后缀有多种，看你保存哪种文件，有 `.R`、`.Rdata` 等后缀，`.R` 一般是保存代码的，`.Rdata` 是保存处理后的数据的。

2.1.10 保存数据

可以使用 `save()` 函数，可以保存多种格式的文件

```
1 > save(b, file="xxx.Rdata") #保存b文件是,，格式为xxx.Rdata
2 > save(b,c, file="xxx.Rdata") #保存b,c文件是,，格式为xxx.Rdata
3 > save.image("xxx.Rdata") #保存所有（你定义的所有变量及数据），文件是,，格式为xxx.Rdata
```

2.2 R 语言语法

2.2.1 函数

R 中含有大量的函数，比如求和函数 `sum()`，求均值 `mean()` 等直接可以拿来使用的函数，但是往往在实际操作过程中，因为需求不同需要自己写函数，就是自定义函数，类似于 C 或 C++ 中的自

定义函数，我们来看一下 R 中如何自定义函数，请看如下代码：

```

1 > #函数名<-function(){.....}
2 > f<-function(a,b)    #写一个求和函数,function()是固定格式
3 + {
4 +   k<-a+b
5 +   return(k)         #返回结果为，如果没有设置返回值，则函数会返回最后一行执行的结果k
6 + }
7 > f(3,4)              #使用函数名加一个括号调用函数
8 [1] 7

```

注：函数可以套函数使用

输出函数

`print()` 函数，输出，与 C 语言中的 `printf()` 函数作用一样

```

1 > print(5)
2 [1] 5

```

安装 R 包

安装 R 包需要使用 `install.packages()` 或者 `BiocManager::install()` 函数安装，`library()` 函数调用，以安装 `openxlsx` 为例，如下：

```

1 安装包代码如下
2 R
3 install.packages("openxlsx")或
4 BiocManager::install("openxlsx")
5 library(openxlsx)

```

文件的读取

在 R 语言中，读取文件是数据分析的第一步。根据文件格式（txt, csv, xlsx）和数据结构的复杂度，我们可以选择不同的函数。以下是几种常用的读取函数及其参数详解：

1. **scan()** 函数：适用于读取简单的数值、字符序列或大规模规则数据。

- **file**: 文件路径字符串。注意 Windows 路径需用 "/" 或 "\"。
- **what**: 指定数据类型，如 `double()`, `integer()`, `character()`。
- **sep**: 分隔符，默认为空字符（空格、制表符、换行符均可）。

```

1 # 读取纯数字
2 x <- scan(file = "data.txt", what = double())

```

```
3 # 读取混合类型（作为字符读入）
4 y <- scan(file = "mixed.txt", what = "")
```

Listing 2.1: scan 函数示例

2. `readline()` 函数：主要用于交互式脚本，从控制台读取单行输入。

- **prompt**: 打印在终端的提示信息。

```
1 name <- readline(prompt = "请输入您的姓名：")
2 print(paste("你好", name))
```

3. `readLines()` 函数：用于以“行”为单位读取文本文件。

- **n**: 读取的行数。
- **encoding**: 字符编码方式（如 "UTF-8"），解决中文乱码的关键。

```
1 # 读取前三行内容
2 txt_content <- readLines("memo.txt", n = 3, encoding = "UTF-8")
```

4. `read.table()` 函数：读取结构化表格数据的“万能钥匙”。

- **header**: 逻辑值，首行是否包含列名。
- **sep**: 字段分隔符。常见："," (CSV), "\t" (Tab 间隔)。
- **na.strings**: 指定代表缺失值的字符。

```
1 # 读取带有表头、逗号分隔、且将视为缺失值的文件"999"
2 my_data <- read.table("survey.txt", header = TRUE, sep = ",", na.strings = "
  999")
```

5. `read.csv()` 函数：`read.table` 的变体，专为逗号分隔文件优化。

```
1 # 默认 header = TRUE
2 df <- read.csv("results.csv")
```

6. `read.xlsx()` 函数：读取 Excel 文件的标准方案（需 `openxlsx` 包）。

```
1 library(openxlsx)
2 excel_data <- read.xlsx("report.xlsx", sheet = 1, startRow = 2)
```

常用读取函数对比表

注意事项： 在读取文件前，建议先使用 `getwd()` 查看当前工作目录，或使用 `setwd()` 设置路径，这样在 `file` 参数中只需写文件名即可，能极大简化代码。

函数	主要用途	返回类型	适用场景
<code>scan</code>	快速读取元素	向量/列表	大规模数值或简单序列
<code>readline</code>	交互式输入	长度为 1 的字符向量	脚本运行中的人工干预
<code>readLines</code>	按行读取文本	字符向量	文本挖掘或非结构化数据
<code>read.table</code>	读取结构化表格	数据框 (data.frame)	绝大多数实验原始数据
<code>read.xlsx</code>	读取 Excel 文件	数据框 (data.frame)	现成的统计报表

文件的输出

在 R 语言中，将处理好的数据或结果保存到本地文件，主要使用以下几种函数：

1. **`write.table()` 函数：**与 `read.table()` 对应，用于将数据框或矩阵输出为结构化文本文件。

- **`x`:** 要写入的对象。
- **`file`:** 保存路径。
- **`append`:** 逻辑值。若为 `TRUE`，则在原文件末尾追加；若为 `FALSE`，则覆盖原文件。
- **`sep`:** 分隔符，如 `sep = ","` 表示保存为 CSV 格式。
- **`row.names/col.names`:** 是否保留行名或列名（通常建议设为 `FALSE` 以保持纯净的数据格式）。

2. **`write()` 函数：**常用于将向量数据以特定列数写入文件。

```
1 # 将字符串写入桌面文件
2 write("Hello, world", "C:/Users/橙/Desktop/juli.txt")
```

3. **`cat()` 函数：**功能强大的输出函数，可以将多个对象拼接并直接写入文件。

- **`file`:** 指定输出文件的路径。
- **`append`:** 控制写入模式。
 - `append = TRUE` (或 `T`): 追加写入，在文件原有内容后添加。
 - `append = FALSE` (或缺省): 覆盖写入，删除原文件内容并重新写入。

```
1 # 追加写入示例
2 cat("hello world", file="C:/Users/橙/Desktop/juli.txt", append=TRUE)
3
4 # 覆盖写入示例
5 cat("hello world", file="C:/Users/橙/Desktop/juli1.txt")
```

4. **`write.csv()` 函数：**专用于输出逗号分隔文件的便捷函数。

- 默认 `sep = ","` 且 `col.names = TRUE`。

5. `save()` / `saveRDS()` 函数：用于保存 R 特有的二进制格式（.RData / .rds），适合保存模型或中间变量。

函数	输出对象类型	主要特点
<code>write.table</code>	数据框/ 矩阵	参数丰富，兼容性最强
<code>write</code>	向量	简单快捷，常用于简单列表
<code>cat</code>	字符串/ 拼接对象	灵活，支持追加/覆盖两种模式
<code>write.csv</code>	数据框	自动化设置，常用于 Excel 交互

输出函数对比总结

温馨提示： 在输出数据到 CSV 时，若不想让第一列出现行索引数字，请务必加上参数 `row.names = FALSE`。

2.2.2 向量 (Vector)

向量是 R 语言中最基本的数据结构，其特点是所有元素必须属于同一类型（数值型、字符型或逻辑型）。

1. 向量的创建

- `c()` 函数：最常用的组合函数（combine），用于连接元素。
- `assign()` 函数：较少用，等同于赋值符号。

```
1 x <- c(1, 3, 5, 7)           # 数值向量
2 y <- c("Apple", "Banana")    # 字符向量
3 assign("z", c(1, 2))         # 等同于 z <- c(1, 2)
```

2. 创建等差序列

- 冒号运算符 (`:`)：生成步长为 1 的序列。
- `seq()` 函数：生成等差序列，可指定步长（by）或长度（length.out）。
- `rep()` 函数：生成重复序列。

```
1 a <- 1:10                     # 1 到 10
2 b <- seq(from=1, to=10, by=2) # 1, 3, 5, 7, 9
3 c <- rep(1:3, times=2)        # 1, 2, 3, 1, 2, 3
4 d <- rep(1:3, each=2)         # 1, 1, 2, 2, 3, 3
```

3. 向量的索引 (Indexing) R 的索引从 1 开始，这与 Python/C 不同。

- 正整数索引：提取对应位置元素。
- 负整数索引：剔除对应位置元素。
- 逻辑索引：提取满足条件 (TRUE) 的元素。
- 名称索引：若向量有 names，可按名提取。

```

1 v <- c(10, 20, 30, 40, 50)
2 v[2]           # 提取第2个: 20
3 v[c(1, 3)]     # 提取第1和第3个: 10, 30
4 v[-1]          # 剔除第1个: 20, 30, 40, 50
5 v[v > 35]      # 提取大于35的: 40, 50

```

4. 向量元素的更改 通过索引选中特定位置的元素，并使用赋值符号 <- 将新值赋给它。

```

1 v <- c(10, 20, 30, 40)
2 v[2] <- 200      # 将第2个元素修改为 200
3 v[v < 35] <- 0    # 将所有小于 35 的元素统一修改为 0
4 v[c(1, 3)] <- c(8, 9) # 同时修改第1和第3个元素

```

5. 向量元素的删除 R 语言中“删除”元素的本质是：创建一个不包含该元素的新向量并覆盖原变量。

- 按位置删除：使用负索引。
- 按值删除：利用逻辑判断结合 != (不等于) 或 %in% (包含于)。
- 全部清空：将变量赋值为 NULL。

```

1 v <- c("A", "B", "C", "D", "E")
2
3 # 1. 删除特定位置元素
4 v <- v[-2]          # 删除第2个元素 "B"
5 v <- v[-c(1, 3)]    # 同时删除第1和第3个元素
6
7 # 2. 按值删除
8 v <- v[v != "E"]     # 提取所有不等于 "E" 的元素，达到删除 "E" 的效果
9
10 # 3. 删除特定范围的元素
11 nums <- 1:10
12 nums <- nums[!(nums %in% c(2, 4, 6))] # 删除 2, 4, 6

```

6. 向量的扩展 可以通过为超出当前长度的索引赋值来动态扩展向量。

```

1 v <- c(1, 2)
2 v[4] <- 10          # v 变为 c(1, 2, NA, 10)，中间会自动补 NA

```

7. 向量化运算 (Vectorized Operation) R 的一大特色是运算符会作用于向量的每一个元素。

```
1 x <- c(1, 2, 3)
2 y <- c(10, 20, 30)
3 x + y          # 结果: 11, 22, 33
4 x * 2          # 结果: 2, 4, 6
```

类别	函数	功能说明
属性查询	length(x)	返回向量的长度（元素个数）
	mode(x)	返回向量的数据类型（如 numeric, character）
	names(x)	获取或设置向量元素的名称
统计汇总	sum(x)	向量元素求和
	mean(x)	向量元素求平均值
	max(x) / min(x)	返回向量中的最大值/ 最小值
	sd(x) / var(x)	返回向量的标准差/ 方差
排序与转换	sort(x)	对向量进行排序（默认为升序）
	rev(x)	将向量元素完全反转
	unique(x)	去除向量中的重复元素，仅保留唯一的项
	order(x)	返回排序后的元素在原向量中的索引位置
搜索与逻辑	which(x == a)	返回符合条件 a 的元素下标（位置）
	which.max(x)	返回最大值所在的第一个位置索引
	is.na(x)	检测缺失值，返回逻辑向量（TRUE/FALSE）
	any() / all()	判断向量中是否存在/ 是否全部为 TRUE
数据清洗	na.omit(x)	移除向量中的所有缺失值 (NA)
	round(x, n)	将数值四舍五入保留 n 位小数

常用向量函数汇总表

注意事项： 如果将不同类型的数据放入同一个向量，R 会进行“隐式转换”（Coercion），转换顺序通常为: Logical < Integer < Double < Character。例如 c(1, "A") 会导致 1 被转换为字符型 "1"。

2.2.3 NA 与 NULL 值

在 R 语言中，NA 和 NULL 都有“空”的含义，但在处理逻辑和数据结构中有着本质的区别。

- 1. NA (Not Available) NA 表示内容缺失。它是一个“占位符”，代表该位置应该有值，但目前不知道是多少。

- **特点:** NA 具有长度（长度为 1），且有类型之分（如 NA_integer_, NA_character_）。
- **逻辑运算:** 任何与 NA 进行的算术运算结果通常仍为 NA（例如 1 + NA = NA）。

2. **NULL (Null Object)** NULL 代表一个空对象。它表示“什么都没有”，连占位符都不是。

- **特点:** NULL 的长度始终为 0，不占用向量空间。
- **用途:** 常用于初始化一个空变量，或者从列表（List）中彻底删除某个元素。

3. **常用检测与处理函数** 注意：NA 不能使用 == 进行判断，必须使用专用函数 is.na()。

```
1 x <- c(1, 2, NA, 4)
2 is.na(x)           # 返回逻辑向量: FALSE FALSE TRUE FALSE
3 anyNA(x)           # 快速检查是否存在缺失值: TRUE
4
5 y <- NULL
6 is.null(y)         # 检查是否为NULL: TRUE
```

4. **向量中的行为对比** 这是理解两者差异最直观的方式：

```
1 # NA 占据位置，作为元素存在
2 v_na <- c(1, NA, 3)
3 length(v_na)       # 结果为 3
4
5 # NULL 不占位置（在组合时被忽略）
6 v_null <- c(1, NULL, 3)
7 length(v_null)     # 结果为 2
```

维度	NA (缺失值)	NULL (空对象)
含义	存在但未知 (Missing value)	不存在该对象 (Undefined)
长度	1	0
类型示例	NA_integer_, NA_character_	唯一的 NULL 类
运算	1 + NA = NA	1 + NULL = numeric(0)
检测函数	is.na()	is.null()
数据框行为	在表格中显示为单元格空白/- NA	无法作为独立列存在

NA 与 NULL 核心差异表

进阶提示： 在进行统计计算（如 `sum()`, `mean()`）时，如果向量包含 `NA`，结果默认会变成 `NA`。此时需要显式设置参数 `na.rm = TRUE` 来剔除缺失值后再计算：`mean(x, na.rm = TRUE)`。

2.2.4 矩阵 (Matrix)

矩阵是一个二维的数据结构，可以看作是排列成行和列的向量。

1. 矩阵的创建 使用 `matrix()` 函数，通过指定数据向量、行数或列数来创建。

- **data:** 包含矩阵元素的向量。
- **nrow / ncol:** 设定的行数和列数。
- **byrow:** 逻辑值。若为 `TRUE`，则按行填充数据；默认为 `FALSE`（按列填充）。

```
1 # 创建一个 2 行 3 列的矩阵
2 m1 <- matrix(1:6, nrow = 2, ncol = 3)
3 # 按行填充
4 m2 <- matrix(1:6, nrow = 2, byrow = TRUE)
```

2. 矩阵的合并与维度命名

- **rbind():** 将多个向量或矩阵按行合并。
- **cbind():** 将多个向量或矩阵按列合并。
- **dimnames:** 设置行名和列名。

```
1 v1 <- c(1, 2); v2 <- c(3, 4)
2 m_row <- rbind(v1, v2) # 行列22
3 colnames(m_row) <- c("Col1", "Col2")
4 rownames(m_row) <- c("Row1", "Row2")
```

3. 矩阵的索引 (Indexing) 使用 [行索引, 列索引] 的格式。

```
1 m <- matrix(1:9, nrow = 3)
2 m[1, 2]      # 提取第 1 行第 2 列的元素
3 m[1, ]      # 提取第 1 行所有元素（返回向量）
4 m[, 2:3]     # 提取第 2 到 3 列所有元素
5 m[-1, ]     # 删除第 1 行后的剩余部分
```

4. 常用矩阵运算函数 矩阵运算在 R 中非常高效。

```
1 t(m)         # 矩阵转置
2 diag(m)      # 提取对角线元素
3 m * m        # 元素级乘法（对应位置相乘）
4 m %*% m      # 线性代数中的矩阵乘法
5 colSums(m)   # 对每一列求和
6 rowMeans(m)  # 对每一行求平均值
```

函数	功能说明	返回值示例
<code>dim(m)</code>	获取矩阵的维度（行数和列数）	<code>c(nrow, ncol)</code>
<code>nrow(m)</code>	单独获取行数	整数
<code>ncol(m)</code>	单独获取列数	整数
<code>attributes(m)</code>	查看矩阵的所有元数据（如维度、行列名）	列表

矩阵核心属性函数表

进阶提示： 1. 如果对矩阵进行索引时只取一行或一列，结果默认会退化（Drop）为向量。如果不希望退化，请使用 `m[1, , drop = FALSE]`。2. 矩阵本质上是一个带有维度属性（`dim`）的向量，因此 `length(m)` 返回的是矩阵中元素的总个数。

2.2.5 列表 (List)

列表是 R 语言中最复杂的数据结构，它可以包含不同类型的对象，且对象之间的长度可以不同。

1. 列表的创建 使用 `list()` 函数创建，建议在创建时为每个成分命名，方便后续调用。

```

1 # 创建包含向量、矩阵和字符串的列表
2 my_list <- list(
3   vec = c(1, 2, 3),
4   mat = matrix(1:4, nrow = 2),
5   msg = "Hello R"
6 )

```

2. 列表的索引 (关键点) 访问列表元素有两种常用的方式，其返回结果完全不同：

- 单方括号 `[ ]`：提取列表的 ** 子集 **，结果仍为一个“小列表”。
- 双方括号 `[[ ]`：提取列表中的 ** 具体内容 **，结果为该位置原始的对象类型。
- 美元符号 `$`：按名称提取内容，效果等同于 `[[ ]`。

```

1 my_list[1]      # 返回一个只含向量的列表
2 my_list[[1]]    # 返回具体的数值向量：1, 2, 3
3 my_list$vec     # 同上，返回数值向量

```

3. 列表元素的更改与添加

```

1 # 修改元素
2 my_list[[3]] <- "Goodbye"
3
4 # 添加新元素
5 my_list$new_item <- c(TRUE, FALSE)

```

```

6
7 # 删除元素：赋值为 NULL
8 my_list$msg <- NULL

```

4. 列表的合并与转换

```

1 # 合并列表
2 combined_list <- c(list1, list2)
3
4 # 将列表转换为向量（若类型一致）
5 v <- unlist(my_list)

```

操作	语法	结果说明
提取子集	<code>L[i]</code>	返回一个列表，包含第 <i>i</i> 个元素
提取内容	<code>L[[i]]</code>	返回第 <i>i</i> 个元素本身的类型（向量/矩阵等）
按名提取	<code>L\$name</code>	与 <code>L[["name"]]</code> 等价，最常用的方式
元素个数	<code>length(L)</code>	返回列表中顶层成分的数量
查看结构	<code>str(L)</code>	极其重要： 显示列表的层级和内容结构

列表核心操作汇总表

进阶提示： 1. 为什么需要双括号？想象列表是一个装着许多盒子的货车，`L[1]` 是卸下了第一辆货车上的第一个“盒子”，而 `L[[1]]` 则是打开了盒子，拿到了里面的“货物”。2. **`str()` 函数：**面对复杂的统计模型输出结果（通常是多层嵌套列表），使用 `str()` 是理清数据脉络的第一步。

2.2.6 数据框 (Data Frame)

数据框是 R 中最常用的数据结构，用于存储表格状数据。

1. **数据框的创建** 使用 `data.frame()` 函数，将多个向量组合成列。

```

1 # 创建一个包含病患信息的数据框
2 patient_id <- c(101, 102, 103)
3 age <- c(25, 42, 37)
4 treatment <- c("A", "B", "A")
5
6 df <- data.frame(ID = patient_id, Age = age, Group = treatment)

```

2. **查看数据框属性** 对于大型数据集，直接输入变量名查看是不现实的。

- `head(df, n)`: 查看前 `n` 行（默认为 6 行）。
- `tail(df, n)`: 查看后 `n` 行。
- `dim(df)`: 获取行数和列数。
- `str(df)`: 查看数据框结构（各列类型及前几个值）。
- `summary(df)`: 获取每列的统计摘要（最大、最小、均值等）。

3. 数据框的索引与提取

```

1 # 1. 访问列
2 df$Age           # 返回 Age 列的向量
3 df[["Age"]]      # 同上
4 df[, 2]          # 提取第 2 列
5
6 # 2. 访问行
7 df[1, ]          # 提取第 1 行
8
9 # 3. 逻辑筛选（极其常用）
10 # 提取 Group 为 "A" 的所有病患行
11 group_a <- df[df$Group == "A", ]

```

4. 数据框的修改与合并

```

1 # 添加新列
2 df$Gender <- c("M", "F", "M")
3
4 # 修改列名
5 colnames(df)[3] <- "Treatment_Group"
6
7 # 合并
8 # rbind(df1, df2) 合并行（列名需一致）
9 # cbind(df1, df2) 合并列（行数需一致）

```

函数	功能说明	备注
<code>nrow(df)</code>	返回行数	对应观察对象数
<code>ncol(df)</code>	返回列数	对应变量数
<code>names(df)</code>	获取列名	等价于 <code>colnames()</code>
<code>subset()</code>	按条件提取子集	<code>subset(df, Age > 30)</code>
<code>with()</code>	简化列名引用	<code>with(df, mean(Age))</code>

数据框处理常用函数表

进阶提示： 1. **字符与因子：**在旧版 R 中，创建数据框时字符会自动转为因子（Factor）。现代版本（R 4.0+）默认不再转换。若需手动控制，可设置 `stringsAsFactors = TRUE`。2. **缺失值处理：**使用 `na.omit(df)` 可以快速删除数据框中所有包含 NA 的行。

2.2.7 因子 (Factor)

因子用于存储分类变量。在 R 内部，因子被存储为整数向量，并带有一个标签（Label）向量。

1. 因子的创建 使用 `factor()` 函数。

- **levels:** 手动指定分类的水平。如果不指定，R 会按字母顺序排列。
- **labels:** 为水平设置更直观的标签。

```
1 # 创建简单因子
2 gender <- factor(c("Male", "Female", "Female", "Male"))
3 # 查看因子水平
4 levels(gender) # 结果为 "Female" "Male"
```

2. 有序因子 (Ordered Factor) 用于有明显等级顺序的数据（如：病情的轻、中、重）。

```
1 status <- c("Low", "High", "Medium", "High", "Low")
2 # 指定等级顺序，否则默认按字母排序 (High, Low, Medium)
3 status_factor <- factor(status,
4                           ordered = TRUE,
5                           levels = c("Low", "Medium", "High"))
```

3. 常用函数与操作

```
1 # 1. 频数统计
2 table(gender)      # 统计每个水平出现的次数
3
4 # 2. 修改因子水平
5 levels(gender) <- c("F", "M") # 将 Female/Male 简写
6
7 # 3. 将连续变量因子化 分组()
8 age <- c(22, 45, 18, 65, 30)
9 age_group <- cut(age, breaks = c(0, 18, 60, Inf),
10                  labels = c("Minor", "Adult", "Senior"))
```

因子相关常用函数表

进阶提示：数值型因子的“陷阱” 如果一个因子是由数字组成的（例如 `f <- factor(c(10, 20, 30))`），直接使用 `as.numeric(f)` 会返回其内部的 ** 层级索引 **（1, 2, 3）而不是原本的数字。正确转换方法：`as.numeric(as.character(f))`。

函数	功能说明	备注
<code>is.factor()</code>	检查对象是否为因子	返回逻辑值
<code>levels()</code>	获取或设置因子的水平	返回字符向量
<code>nlevels()</code>	返回因子的水平个数	等价于 <code>length(levels())</code>
<code>as.factor()</code>	将其他类型强制转换为因子	不支持指定顺序
<code>gl()</code>	生成规则的因子序列	<code>gl(n, k)</code> 生成 n 个水平, 每水平重复 k 次

2.2.8 常用数学与统计函数

R 语言内置了极其丰富的数学与统计工具, 能够高效处理向量化数据。

1. 基本数学函数 这些函数通常作用于向量中的每一个元素。

```

1 abs(-5)           # 绝对值: 5
2 sqrt(16)          # 开平方: 4
3 exp(1)            # 指数函数: 2.718...
4 log(100, base=10) # 对数函数, 默认以 e 为底
5 round(3.1415, 2)  # 四舍五入保留两位小数: 3.14
6 ceiling(3.1)      # 向上取整: 4
7 floor(3.9)        # 向下取整: 3

```

2. 统计汇总函数 用于描述性统计分析。注意: 若数据中包含 NA, 需添加 `na.rm = TRUE`。

```

1 x <- c(1, 5, 10, 20, NA)
2 sum(x, na.rm = TRUE)    # 求和
3 mean(x, na.rm = TRUE)   # 算术平均值
4 median(x, na.rm = TRUE) # 中位数
5 var(x, na.rm = TRUE)    # 方差
6 sd(x, na.rm = TRUE)     # 标准差
7 range(x, na.rm = TRUE)  # 极差 (返回最小值和最大值)
8 quantile(x, 0.75, na.rm = TRUE) # 计算 75% 分位数

```

3. 概率分布函数 (以正态分布为例) R 中对每种分布都有四类函数, 其前缀分别为:

- **d** (density): 密度函数/ 概率质量函数。
- **p** (probability): 累积分布函数 (左尾概率)。
- **q** (quantile): 分位数函数 (p 的反函数)。
- **r** (random): 生成符合该分布的随机数。

```

1 rnorm(100, mean = 0, sd = 1) # 生成 100 个标准正态分布随机数
2 pnorm(1.96)                  # 标准正态分布下,  $Z < 1.96$  的概率 约 ( 0.975 )
3 qnorm(0.95)                  # 找到对应 95% 累积概率的 Z 值 约 ( 1.64 )

```

分布类型	R 函数名后缀	典型应用场景
正态分布 (Normal)	<code>norm</code>	绝大多数连续变量分析
T 分布 (Student's t)	<code>t</code>	小样本均值检验
二项分布 (Binomial)	<code>binom</code>	患病/未患病等二分类实验
泊松分布 (Poisson)	<code>pois</code>	罕见事件发生次数统计
卡方分布 (Chi-squared)	<code>chisq</code>	计数资料的关联性检验

常用分布函数名对照表

进阶提示： 在进行统计绘图或模拟实验时，如果希望结果具有可重复性，请务必在生成随机数前使用 `set.seed(数值)` 函数设置随机数种子（如 `set.seed(123)`）。

2.3 控制流与自定义函数

控制流用于管理代码的执行逻辑，而函数则将一系列操作封装成一个可重复调用的单元。

1. 条件判断 (If-Else) 用于根据逻辑条件的真假执行不同的代码块。

```
1 x <- 85
2 if (x >= 90) {
3     print("优秀")
4 } else if (x >= 60) {
5     print("合格")
6 } else {
7     print("不合格")
8 }
9
10 # 向量化版本: ifelse() 函数 极其常用()
11 # 语法: ifelse(test, yes, no)
12 status <- ifelse(x >= 60, "Pass", "Fail")
```

2. 循环结构 (Loop)

- **for 循环：**遍历向量或列表中的每一个元素。
- **while 循环：**当条件满足时持续运行。

```
1 # for 循环示例
2 for (i in 1:5) {
3     print(i^2)
4 }
5
```

```
6 # 批量处理文件名示例
7 files <- c("sample1.csv", "sample2.csv")
8 for (f in files) {
9     # data <- read.csv(f)
10    # ... 处理代码 ...
11 }
```

3. 自定义函数 (Function) 函数允许你将复杂的逻辑打包。R 函数会自动返回最后一行表达式的结果。

```
1 # 定义函数语法: 函数名 <- function参数(1, 参数2, ...) { 逻辑体 }
2 calc_bmi <- function(weight, height) {
3     bmi <- weight / (height^2)
4     return(bmi) # 显式返回, 也可以省略直接写 bmi
5 }
6
7 # 调用函数
8 my_bmi <- calc_bmi(70, 1.75)
```

关键字	功能说明	应用场景
if / else	标准条件分支	针对单个逻辑判断
ifelse()	向量化条件判断	推荐: 用于对数据框整列进行分类
for	固定次数循环	遍历已知长度的对象
break	强制跳出当前循环	满足特定错误或条件时停止
next	跳过本次循环, 进入下一次	过滤不符合条件的样本
return()	指定函数返回值	提前结束函数或明确输出结果

控制流常用关键字与技巧表

进阶提示: 避免“显式循环” R 语言是向量化语言。在处理大型数据框时, 尽量使用 ifelse() 或 apply 系列函数 (如 lapply, sapply), 而不是直接写 for 循环。这不仅能让代码更简洁, 还能显著提升运行速度。

Chapter 3

数据分析

3.1 数据清洗与转换 (dplyr & tidyr)

在 R 语言的的实际应用中，原始数据往往是“脏”的或格式不规范的。`tidyverse` 生态系统中的 `dplyr` 和 `tidyr` 提供了一套高度一致、语义化的工具。

3.1.1 dplyr 的核心动作 (Verbs)

`dplyr` 包通过几个核心动词函数，覆盖了大部分数据转换需求：

1. `filter()`：按行筛选数据。
2. `select()`：按列选取变量。
3. `mutate()`：衍生新列。
4. `arrange()`：数据排序。
5. `group_by()` 与 `summarise()`：分组聚合。

3.1.2 tidyr 的数据重塑

- `pivot_longer()`：将宽格式数据转换为长格式。
- `pivot_wider()`：将长格式数据转换为宽格式。

3.1.3 常用清洗函数对比总结

3.2 数据可视化 (ggplot2)

`ggplot2` 是 R 语言中最强大的可视化包。其核心逻辑是通过加号 (+) 将不同的图层叠加在一起。

函数	功能描述	SQL/Excel 对应
<code>filter()</code>	筛选满足条件的观察行	Filter / WHERE
<code>select()</code>	选取或剔除指定的变量列	Select / Column hide
<code>mutate()</code>	新增列并保留原数据	Calculated Field
<code>group_by()</code>	将数据按类别分组	GROUP BY
<code>summarise()</code>	将多行数据压缩为一行汇总结果	Pivot Table / Sum
<code>inner_join()</code>	根据键值合并两个数据框	VLOOKUP / JOIN

3.2.1 ggplot2 的基础语法结构

一个基本的绘图模板如下：

```
1 ggplot(data = <DATA>) +  
2   <GEOM_FUNCTION>(mapping = aes(<MAPPINGS>)) +  
3   <THEME_FUNCTION>()
```

1. **数据 (Data):** 必须是一个数据框 (`data.frame`)。
2. **映射 (Mapping):** 使用 `aes()` 函数定义，指定变量如何映射到视觉属性（如 x 轴、y 轴、颜色 `color`、形状 `shape`）。
3. **几何对象 (Geoms):** 决定图表类型（点、线、柱等）。

3.2.2 常见图表类型 (Geoms)

根据研究目的选择不同的几何对象函数：

- **散点图:** `geom_point()` ——用于观察两个连续变量的关系。
- **箱线图:** `geom_boxplot()` ——用于观察变量的分布及异常值。
- **条形图:** `geom_bar()` 或 `geom_col()` ——用于展示分类数据的频数或数值。
- **折线图:** `geom_line()` ——常用于时间序列数据。
- **直方图:** `geom_histogram()` ——观察单变量的分布密度。

3.2.3 图形美化：颜色、标签与主题

通过叠加额外的图层，可以让图表达达到出版标准。

```
1 df %>%
2   ggplot(aes(x = weight, y = height, color = group)) +
3   geom_point(size = 3, alpha = 0.7) +
4   labs(
5     title = "Weight vs Height",
6     x = "Weight (kg)",
7     y = "Height (cm)",
8     color = "Group Name"
9   ) +
10  theme_bw() # 使用经典白底黑框主题
```

3.2.4 分面 (Faceting)

分面是 ggplot2 最强大的功能之一，可以根据某个分类变量快速生成多张子图。

- `facet_wrap(~variable)`: 按一个变量自动排列子图。
- `facet_grid(rows ~ cols)`: 按行和列两个变量生成矩阵子图。

3.2.5 常用外观调整函数对照表

类别	函数	功能说明
坐标轴范围	<code>xlim()</code> , <code>ylim()</code>	设置 X 或 Y 轴的起止点
颜色比例	<code>scale_color_manual()</code>	手动指定离散变量的颜色（如实验组红色，对照组蓝色）
坐标翻转	<code>coord_flip()</code>	将 X 轴与 Y 轴对调（常用于条形图）
预设主题	<code>theme_minimal()</code>	极简主题，去除冗余网格线
保存图表	<code>ggsave()</code>	将最后生成的图表保存为 pdf, png 或 tiff 格式

注意事项： 1. 注意 `color`（轮廓色/点色）与 `fill`（填充色）的区别。柱状图的颜色通常使用 `fill`。2. 多个图层之间必须使用 `+` 连接，且 `+` 必须写在行末，不能写在行首。3. 如果在 `aes()` 内部设置颜色，它是根据数据自动映射；如果在 `aes()` 外部设置颜色（如 `geom_point(color = "blue")`），则是统一指定颜色。

3.3 统计检验基础

R 语言提供了强大的内置函数，用于执行常见的参数化与非参数化检验。在执行检验前，通常需要确保数据满足正态性 (`shapiro.test()`) 和方差齐性 (`bartlett.test()`)。

3.3.1 均值比较 (T-Test & ANOVA)

1. **T 检验 (t-test)** 用于比较两组数据均值是否存在显著差异。

- **独立样本 T 检验**：比较两组独立受试者。
- **配对样本 T 检验**：比较同一受试者治疗前后的变化。

```
1 # 独立样本: y 是连续变量, group 是二分类因子
2 t.test(y ~ group, data = df)
3
4 # 配对样本
5 t.test(df$before, df$after, paired = TRUE)
```

2. **方差分析 (ANOVA)** 用于比较三组及以上数据的均值差异。

```
1 # 1. 拟合线性模型
2 fit <- aov(weight ~ group, data = df)
3
4 # 2. 查看 ANOVA 表
5 summary(fit)
6
7 # 3. 事后两两比较 (Tukey HSD)
8 TukeyHSD(fit)
```

3.3.2 相关性与回归分析

1. **相关性分析** 计算变量间的相关系数 (Pearson 或 Spearman)。

```
1 cor(df$age, df$blood_pressure, method = "pearson")
2 cor.test(df$age, df$blood_pressure) # 包含显著性检验
```

2. **线性回归 (Linear Regression)** 用于探究自变量 (x) 对因变量 (y) 的影响。

```
1 # 拟合模型: y = ax + b
2 model <- lm(y ~ x1 + x2, data = df)
3
4 # 查看结果 (包括系数、方和 R P 值)
5 summary(model)
6
7 # 提取残差
8 residuals(model)
```

3.3.3 常用统计函数速查表

统计需求	R 函数	适用场景
正态性检验	<code>shapiro.test()</code>	判断数据是否符合正态分布 ($P > 0.05$)
二组均值	<code>t.test()</code>	两组连续变量比较
二组中位数	<code>wilcox.test()</code>	非正态分布数据的两组比较
多组均值	<code>aov()</code>	三组及以上均值比较 (ANOVA)
分类变量检验	<code>chisq.test()</code>	观察频数与期望频数的差异 (卡方检验)
线性模型	<code>lm()</code>	回归分析与预测

注意事项：关于 P 值 1. 在 R 的输出结果中，星号通常代表显著性水平：*** ($P < 0.001$), ** ($P < 0.01$), * ($P < 0.05$), . ($P < 0.1$)。2. 对于回归模型，在使用 `summary(model)` 后，重点关注 `Coefficients` 表中的 `Pr(>|t|)` 列以及底部的 `Adjusted R-squared` (调整后 R 方)。